2021 IEEE International Conference on Pervasive Computing and Communications Workshops and other Affiliated Events (PerCom Workshops) | 978-1-6654-0424-2/21/$31.00 ©2021 IEEE | DOI: 10.1109/PERCOMWORKSHOPS51409.2021.9430865

Keyword Spotting for Industrial Control using Deep Learning on Edge Devices

Fabian Hölzke, Hameem Ahmed, Frank Golatowski, Dirk Timmermann

Institute of Applied Microelectronics and CE
University of Rostock,
Rostock, Germany
Email: fabian.hoelzke2@uni-rostock.de

***Abstract*—Spoken commands promise unique advantages for the control of industrial machinery. Operators are enabled to keep their eyes on safety critical aspects of the process at all times and are free to use their hands in other parts of the process, instead of remote control. Current keyword spotting systems are prone to misunderstanding spoken utterances, especially in noisy environments, and are commonly deployed as non-realtime cloud services. Consequently, these systems can not be trusted with safety critical industrial control. We adapt a DS-CNN and a CNN for keyword spotting and use augmented training data, including real industrial noise, to increase their robustness. Furthermore, we apply post-training quantization and analyze the performance of both networks using multiple embedded systems, including a Google Edge TPU. We carry out a systematic analysis of accuracies, memory footprint and inference times using different combinations of data augmentations, hardware platforms, and quantizations. We show that augmented training data increases the inference accuracy in noisy environments by up to 20 %. Among others, this is demonstrated using an integer quantized network with a memory footprint of 0.57 MByte, reaching inference speeds of less than 5 ms on an embedded CPU and less than 1 ms on the Edge TPU. The results show that keyword spotting for industrial control is feasible on embedded systems and that the training data augmentation has a significant impact on the robustness in challenging environments.**

***Index Terms*—Speech Detection, Industrial Control, Edge Devices, DS-CNN, Keyword Spotting**

I. Introduction

Voice commands are increasingly common in consumer electronics. Home assistants like Google Home, Siri or Alexa offer a natural voice interface to digital services and home automation with an almost non-existing entry barrier or learning curve for the user. Cars offer voice interfaces to control air conditioning or media playback. However, these services are not safety critical and offload voice processing to a cloud service which raises privacy concerns. They also offer limited robustness and command variety when used offline. Voice interfaces in industrial environments are already used in warehouse logistics where human workers receive and acknowledge picking orders. The use of voice interfaces to control heavy machinery is not yet common and introduces specific challenges:

Missed or falsely interpreted user input results in inefficient use and frustration by the human operator and could at worst lead to catastrophic failures involving the machinery and its environment. Therefore, keyword detection needs to be as accurate as possible. Safety critical control also requires that the delay between the spoken voice command and the correct inference of the keyword needs to stay within predictable bounds. Keyword spotting as a cloud service can not ensure this due to the non-deterministic network communication. Network infrastructure within a facility might enable deterministic communication to a local keyword spotting service, especially considering upcoming concepts regarding the integration of time sensitive networking within 5G networks. However, this requires specialized infrastructure that is costly and not yet commonplace. What is left is the integration of the whole keyword spotting mechanism within an embedded device that is held or worn by the machine operator. Consequently, the mechanism needs to be lightweight regarding its memory footprint as well as computational load. The industry is already adapting to the use of smart devices as a display and interface for the operator. A suitable keyword spotting scheme could be integrated into these devices. This would further enhance the existing control solutions with a hands-free interface that allows the operator to keep their eyes on the safety critical aspects of the machinery.

The Tensor Processing Unit (TPU) is an application-specific integrated circuit developed by Google for AI acceleration [1]. The recently introduced Edge TPU aims to accelerate neural network inference in existing edge devices [2]. The host system can offload the inference workload to the TPU and receives results with reduced latency, compared to inference on the local CPU. This allows for more complex and therefore potentially more robust keyword detection in otherwise limited edge devices.

This work aims to apply deep-learning for robust voice command recognition in an industrial setting. The recognition task will be executed on local and possibly wearable hardware and shall be robust against background noise. We present a data augmentation approach using real factory noise as well as artificial noise for increased inference robustness and test the method using a state-of-the-art neural network architecture for keyword spotting. In order to verify the applicability of this approach in an embedded system, we apply different post-training quantization schemes, analyze the resulting models, and compare their performance an a Raspberry Pi 3 and a

Coral Dev board CPU as well as the Coral Dev board using the Edge TPU.

This paper is structured as follows: Section II describes related work regarding keyword spotting on embedded devices. The dataset and the data augmentation scheme as well as the audio feature generation is presented in Section III. Following, two inference architectures are outlined in IV. The evaluation approach is described in Section V and the experimental results are presented in Section VI. Final conclusions are drawn in Section VII.

II. Related Work

Keyword Spotting Systems (KWSs) have traditionally been realized using Hidden Markov Models (HMMs) and Viterbi decoding. These approaches have been found to perform decently in terms of accuracy, however, their large training and inference times have hindered mainstream adoption on constrained hardware [3]. Further developments show that a KWS based on Deep Neural Networks (DNNs) outperforms HMM-based models while being memory efficient as it requires a less complex inference pipeline at higher inference speeds [4].

Compared to DNNs, the more advanced Convolutional Neural Networks (CNNs) use fewer overall parameters and are still applicable to constrained hardware while improving the false rejection rate [5]. In contrast to DNNs, CNNs can easily observe the time and spectral domain features by using an image of the audio features in the form of a spectrogram, preserving the local temporal and spectral correlations for the input speech features [6]. In contrast, a CNN architecture using the audio waveform directly is presented in [7].

The architecture Xception was proposed for efficiently implementing the convolution operations by using depth-wise separable convolutions. The idea was to decompose the standard 3D convolutions into 2D convolutions over each input channel, followed by point-wise (1D) convolutions to combine the channel outputs onto a new channel space in the depth dimension. This publication went on to propose the new architecture as a suitable replacement for the standard 3D convolution operation. The depth-wise variant achieves comparable performance but with less network parameters and therefore, less computations [8].

This study was followed by the MobileNets architecture which demonstrated the use of depth-wise convolutions in achieving compact architectures in the field of computer vision. Their results showed similar accuracies to existing state-of-the-art architectures, but at a fraction of the computational and size requirements [9].

An architecture that focuses on KWS for microcontrollers is presented in [10]. The authors present an analysis of the existing state-of-the-art solutions and introduce a Depth-Wise Separable Convolutional Neural Network (DS-CNN) for the keyword spotting task. This DS-CNN architecture is largely influenced by the MobileNets architecture. It is shown that the implementation of 3D convolutions using a DS-CNN architecture allows for creating much deeper networks, while still allowing for the model to require a fraction of the size and computational requirements. Furthermore, a 94.5% accuracy is achieved, which is ca. 10% better than a DNN with the same number of parameters [11]. The DS-CNN architecture was further improved upon in later research [12]. In contrast to the direct recognition of a fixed set of keywords, a generalized approach to speech recognition is presented in [13].

The model benchmarking done in our work, as well as several of the above-mentioned studies, uses the Google Speech Commands dataset [14].

III. Data Preparation

The Google Speech Commands dataset v2 is specially designed for the task of keyword spotting. The author has taken care of limiting the vocabulary, as well as the duration for each individual file in the dataset. Because of this, it is the dataset of choice for this work.

The preparation of this dataset is threefold: First, the raw data is inspected and filtered as described in the following paragraph. Second, the data is augmented to increase the KWS robustness. Lastly, we describe the extraction process for the features that are used as inputs for the KWS.

Manual inspection of the dataset revealed that about 90 % of the samples are exactly one second long, while the remaining samples are shorter and vary in length. As these shorter samples tend to be cut off mid-utterance or are otherwise of low quality, we choose to discard them and limit our considerations to samples of exactly one second in duration. Additionally, we are concerned with the control of industrial machinery through speech. Therefore, we aim to discriminate the following list of keywords with samples of one second duration: *yes*, *no*, *on*, *off*, *right*, *left*, *up*, *down*, *stop*, *go*, *forward*, *backward* and *follow*. The remaining 22 keyword labels are unlikely to be useful for this task and are unified under the label *unknown_word*, resulting in almost 60.000 samples for this label. A *silence* label is introduced to distinguish between spoken words and background noise. The data for *silence* is generated by the extraction of 30.000 randomly selected one-second intervals of the 'noise' samples that are included in the Google Speech Commands dataset. It is therefore assumed that unknown speech is twice as likely as silence in a real-word scenario. The data is divided into a train-test-validate split of 70:20:10.

We employ various combinations of the following data augmentation types to investigate their respective influence on the robustness of the KWS in an industrial environment:

- Time: A time stretch of up to $\pm$150 ms.
- Freq: A frequency shift of up to 2 semitones.
- Generic noise: An addition of random noise samples from the datasets background noise files, with an amplitude scaling of 0.3 to 0.4.
- Factory noise: An addition of random noise samples from a factory noise file, with an amplitude scaling of 0.4 to 0.5.

Each augmentation is used to produce a new augmented dataset based on the clean (non-augmented) dataset. The

ranges for amplitude scaling are chosen to level the volume of the speech samples and the noise samples to a subjectively realistic and discernable proportion. This is especially necessary in the case of the factory noise, because it was recorded and mastered separately from the speech commands, and in the case of artificial noise, because it is disproportionally loud. The exact value for the abovementioned parameter ranges of augmentation is decided randomly for each sample. For the generic noise augment, real-live noise recordings are chosen with double the probability compared to the artificial noise samples of the dataset. This is done to keep the scenario realistic and lessen the impact of the dis-proportionally intense distortion by the artificial noise. The factory noise is recorded in an assembly hall featuring the operation of heavy machinery, colliding metal and distorted speech in the background. This file is not part of the Google Speech Commands dataset.

We use a modified approach of [11] to extract the Mel-Frequency Cepstral Coefficients (MFCCs) over frames of the raw audio signal as inputs for the classifier training, testing and validation. The feature calculation is done using a sliding window of 40ms length with a stride of 20ms, resulting in 49 frames for a one-second sample. It has been shown that a higher number of MFCC features have little influence on model accuracy, as most of the relevant information exists in the lower order coefficients [15]. Therefore, in contrast to [11], we reduce the number of MFCC features from 40 to 20. This leads to a reduction in the input size and ultimately translates to less computations for inference. The resulting model input has the dimensions 49 $\times$ 20 (Frames $\times$ Features).

IV. Inference architecture

We compare two model architectures: a DS-CNN and a CNN. The CNN architecture is loosely inspired by [5]. It consists of three convolutional layers, followed by a max pooling layer with a 2×2 kernel, a drop out of 50 %, and finally two dense fully connected layers of size 64 and 32, respectively. All three of the convolutional layers have a 3×3 kernel size, while the number of kernels (or filters) is 16, 32 and 64. The kernel stride is 1×1 for each convolutional layer. The total number of trainable parameters for the CNN model architecture is 1,008,975.

The DS-CNN architecture we employ is based on the MobileNet implementation [9] which is also used in [11]. However, the depth of the model is slightly shallower due to the smaller size of the input features. This architecture uses a full (not separated) 3×3 convolution for the first layer and nine subsequent depth-wise separated convolutional layers, followed by an average pooling layer, and finally a fully-connected dense layer. This model consists of a total of 444,751 parameters, out of which 437,711 are trainable. This is less than half the number of parameters compared to the CNN model described previously. The full architecture is listed in Table I.

Both architectures use the commonly applied rectified linear unit (ReLU) function for activation between layers and a final SoftMax output layer for multi-class prediction.

TABLE I
THE DS-CNN ARCHITECTURE WITH *Conv* DENOTING A CONVOLUTIONAL LAYER, *D-Conv* A DEPTH-WISE CONVOLUTION AND *P-Conv* A POINT-WISE CONVOLUTION

Layer	Filters	Kernel shape	Kernel stride	Output Shape
Input	-	-	-	49×20×1
Conv	32	3×3	2×2	25×10×32
D-Conv-1	-	3×3	1×1	25×10×32
P-Conv-1	64	1×1	1×1	25×10×64
D-Conv-2	-	3×3	2×2	13×5×64
P-Conv-2	128	1×1	1×1	13×5×128
D-Conv-3	-	3×3	1×1	13×5×128
P-Conv-3	128	1×1	1×1	7×3×128
D-Conv-4	-	3×3	2×2	7×3×128
P-Conv-4	256	1×1	1×1	7×3×256
D-Conv-5	-	3×3	1×1	7×3×256
P-Conv-5	256	1×1	1×1	7×3×256
⋮	⋮	⋮	⋮	⋮
D-Conv-9	-	3×3	1×1	7×3×256
P-Conv-9	256	1×1	1×1	7×3×256
Average-Pooling	-	2×2	-	3×1×256
Fully-Connected	-	-	-	32
SoftMax	-	-	-	15

V. Experimental setup

We focus on the comparison of the accuracy and computational performance of both model architectures and the augmentation strategies. Additionally, we perform post-training quantization and compare the resulting model size, inference time and accuracy. The experiments include quantizations to half-precision floating point (float-16) and 8-bit integer, including input and output layers (full-int).

The models are created using the Keras Functional API and are converted to TensorFlow Lite and to the quantized variants of the TensorFlow Lite models.

We compare the inference time on three embedded computing platforms:

- A Raspberry Pi 3, employing a BCM2837 SoC with an ARM Cortex-A53 at 1.2 GHz
- A Coral Dev Board with an NXP i.MX 8M SoC that features an ARM Cortex-A53 at 1.5 GHz
- A Coral Dev Board with the Google Edge TPU co-processor

The average inference time of one sample is calculated by measuring the inference time of a batch of 1000 random samples on the respective hardware platforms. The MFCC input features are pre-calculated, in order to highlight the influence of model architecture and quantization. The input data is not quantized and represented as single precision floats. Consequently, the experiments involve an additional quantization and de-quantization step at the beginning and end of the inference pipeline. This operation is mapped to the CPU of the Coral Dev Board in all experiments involving the Edge TPU. We expect that this will have no significant impact on the comparability of the hardware platforms, because the number

of computations for the actual inference far outweigh the initial and final (de-)quantization steps.

The following sub-sections provide more details on the aspects of model training and choice of augmentation combinations.

A. Model Training

All models are trained with the cross-entropy loss function and Adam as the learning optimization algorithm [16]. The batch size is set to 64, as smaller values have been shown to improve model performance [17]. The total number of training iterations used in all experiments is set to 32,915. This results in ca. 2.11M training samples, which is slightly more than the 2.00M that are used in [11]. The number of epochs varies: The non-augmented dataset consists of $n_{clean} = 125,394$ samples. The augmentation strategy results in a dataset size that is multiplied by the number of augmentations included. That is, including k augmentation types (e.g. noise, time-shift, ...) result in $k * n_{clean}$ samples. Therefore, the number of epochs is varied to keep the total number of training iterations fixed and comparable to [11] in all experiments. The resulting maximum number of epochs is 12, when no augmentations are used. We exclude the combination of all 5 augmentations to reduce the maximum number of epochs from 60 to 12. The goal here is to ensure the comparability of the models trained with a varying number of augmentation types.

B. Pre-Selection of Augmentation Combinations

In order to reduce the total amount of augmentation combinations in the following evaluations, we limit the set of experiments to the most promising combinations. For this, a total of 15 augmentation combinations are used to train the CNN model. The model is then tested with the clean dataset, as well as the generic noise and factory noise augments. The accuracy scores are then averaged. We select the 4 best performing augmentation combinations and include the clean dataset for reference as listed in Table II.

TABLE II
THE BEST PERFORMING AUGMENTATION COMBINATIONS AND THE CLEAN DATASET USED TO TRAIN THE CNN ARCHITECTURE. SORTED BY AVERAGE TEST ACCURACY USING CLEAN, GENERIC AND FACTORY NOISE DATASETS.

Training data combination	Accuracy [%]
clean, generic noise, factory noise, freq	88.20
clean, generic noise, factory noise	87.90
clean, generic noise, factory noise, time	87.40
clean, factory noise	83.14
⋮	⋮
clean	73.29

VI. EXPERIMENTAL RESULTS

Table III lists the resulting model sizes before and after conversion from Keras to TensorFlow Lite and of the quantizations. The conversion to TensorFlow Lite results in a model that is approximately one third of the original size. As expected, the quantization to float-16 approximately halves the model size and the quantization to 8-bit integer results in a further reduction to about halve the size.

TABLE III
MODEL SIZE DEPENDING ON ARCHITECTURE AND QUANTIZATION.

Architecture	Format	Quantization	Size [Mbyte]
CNN	Keras	none	12.17
	TensorFlow Lite	none	4.04
		float-16	2.02
		full-int	1.02
		full-int-tpu	1.07
DS-CNN	Keras	none	5.66
	TensorFlow Lite	none	1.73
		float-16	0.88
		full-int	0.57
		full-int-tpu	0.61

The following sub-sections describe the results regarding inference speed and accuracy in more detail.

A. Inference Speed

The inference speeds for the hardware platforms and model architectures using the different quantizations are listed in Table IV. A visualization of the inference speeds is provided in Fig. 4. Here, the timings are converted to the theoretical measure of inferences per second for easier comparison.

The graphs show that using float-16 quantization has no clear effect on inference time. In contrast to this, full-int quantization shows a significant effect. The most impact was observed when using the DS-CNN architecure: On the Rapsberry Pi 3 (BCM2837), the full-int quantization results in a decrease in inference time by 20 % and 28 % for the CNN and DS-CNN architecture, respectively. Using the Coral Dev Board CPU (i.MX 8M) with an integer quantized model results in a reduction in inference time by 10 % and 40 % for the CNN and DS-CNN architecture, respectively. The Edge TPU further decreases the average inference time by 62 % and 86 %, respectively, compared to the inference speed using full-int quantization on the Coral Dev Board.

TABLE IV
AVERAGE TIME SPENT ON THE INFERENCE OF ONE SAMPLE; SORTED BY QUANTIZATION AND HARDWARE PLATFORM

Architecture	Quantization	Inference time [ms]		
		BCM2837	i.MX 8M	Edge TPU
CNN	none	14.39	9.21	
	float-16	14.39	9.03	
	full-int	11.57	8.25	3.15
DS-CNN	none	9.29	6.87	
	float-16	9.43	6.31	
	full-int	6.67	4.10	0.57

The results show that the Edge TPU far outperforms the embedded CPUs, especially with the DS-CNN architecture. This suggests that much more complex DS-CNNs are feasible when inference is offloaded to an embedded Edge TPU.

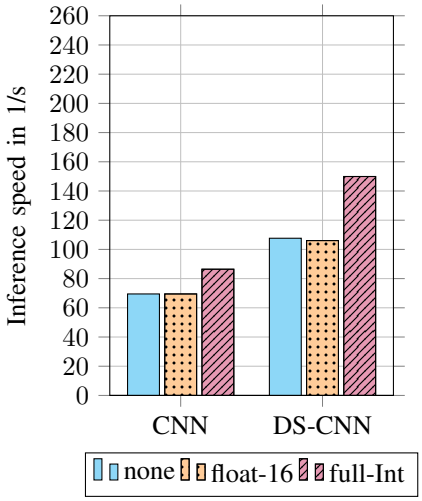

Fig. 1. Raspberry Pi 3 CPU (BCM2837)

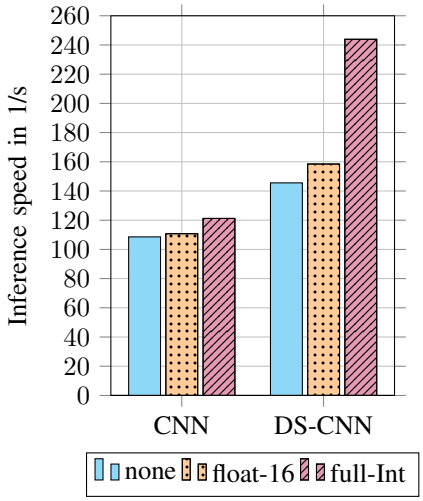

Fig. 2. Coral Dev Board CPU (i.MX 8M)

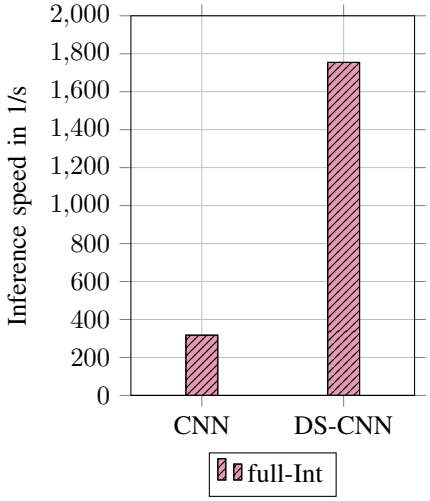

Fig. 3. Google Edge TPU

Fig. 4. Inference speed of the CNN and DS-CNN architectures at different post-training quantizations.

A closer look into the Cortex-A53 that is used in both hardware platforms offers a hint regarding the reason for the CPU timings. The Cortex-A53 implements the ARMv8.0-A architecture, that enables the simultaneous execution of multiple instructions based on the width of the used data types (NEON), as is described in [18], the ARM Compiler armcc User Guide. This means that, e.g. within one clock cylce, four 8-bit integers can be processed simultaneously instead of a single 32-bit floating point number. Furthermore, the ARMv8.0-A architecture does not support the processing of 16-bit floating points, which necessitates the implementation of the Armv8.2-FP16 extension. Instead, half-precision floats are handled as full 32-bit floats.

TABLE V
THE PERFORMANCE OF DIFFERENT AUGMENTATION COMBINATIONS AND ARCHITECTURES, TESTED WITH CLEAN DATA AND SORTED BY ACCURACY.

Architecture	Training data combination	Accuracy [%]
DS-CNN	clean, generic noise, factory noise, freq	96.18
DS-CNN	clean, generic noise, factory noise, time	96.05
DS-CNN	clean, generic noise, factory noise	95.99
DS-CNN	clean, factory noise	95.68
DS-CNN	clean	94.82
CNN	clean, generic noise, factory noise, freq	92.45
CNN	clean, factory noise	91.90
CNN	clean, generic noise, factory noise, time	91.62
CNN	clean, generic noise, factory noise	91.35
CNN	clean	91.09

B. Accuracy

In this section we present the influence of training data augmentations in two test cases: The first case uses clean test data and the second case uses test data that is augmented by factory noise. The results are listed in Tables V and Table VI, respectively. The influence of quantization on accuracy is presented at the end of this section.

The data for clean, non augmented, test data in Table V shows that the DS-CNN outperforms the CNN with every combination of training data augments and clean training data. The worst performing DS-CNN is ahead of the best CNN by 2,37 % and the best DS-CNN is 3,73 % ahead. This shows that the training data augments improve the performance on clean test data. Also, the architectural change to a DS-CNN yields even more accuracy improvements than the data augments.

Table VI shows the results for test data that is augmented by factory noise. The DS-CNN is again superior to the CNN with every combination of training data augmentation, with a 4.40 % gap between the best performing models of each architecture. Comparing this to the smaller 3.73 % gap with clean test data suggests that the DS-CNN has a superior ability to adapt to noisy data. The combinations of augmented training data have the same ranking for the DS-CNN in both test cases. One may assume that the inclusion of factory noise, generic noise and frequency shifted data is superior to factory noise alone. However, the improvement by the additional augments amounts to only 0.21 %. No such pattern is visible for the CNN architecture. Both DS-CNN and CNN models that are trained without any data augments perform significantly worse compared to the augmented variants. However, the DS-CNN still shows a 4.29 % improvement compared to the CNN.

In addition to the previously mentioned experiments, we tested the influence of post-training quantization on accuracy. The resulting differences are marginal: In order to test the effect of quantization, the best performing CNN and DS-CNN model from the training augmentation experiments was selected. These models were then quantized and tested with the clean and noise-augmented test data. The highest effect, with a drop in accuracy of 0.48 %, is seen with full-integer quantization on the CNN model in the factory noise test case. All other tests with factory noise and clean test data, using

TABLE VI
THE PERFORMANCE OF DIFFERENT AUGMENTATION COMBINATIONS AND ARCHITECTURES, TESTED WITH DATA THAT IS AUGMENTED WITH FACTORY NOISE AND SORTED BY ACCURACY.

Architecture	Training data combination	Accuracy [%]
DS-CNN	clean, generic noise, factory noise, freq	91.32
DS-CNN	clean, generic noise, factory noise, time	91.20
DS-CNN	clean, generic noise, factory noise	91.18
DS-CNN	clean, factory noise	91.11
CNN	clean, factory noise	86.92
CNN	clean, generic noise, factory noise, freq	86.46
CNN	clean, generic noise, factory noise	85.99
CNN	clean, generic noise, factory noise, time	84.69
DS-CNN	clean	71.25
CNN	clean	66.96

the remaining quantizations for the CNN and all DS-CNN quantizations, resulted in changes of less than 0.10 %.

It seems that a deeper DS-CNN architecture, using less than half the number of trainable parameters of the CNN and thrice the number of convolution layers, is less susceptible to noisy data inputs as well as quantization noise.

VII. CONCLUSION

In this paper, we describe a KWS for industrial control and address the challenges of robust and timely inference of spoken commands on an embedded system. We show that a deep DS-CNN architecture, trained with augmented data, improves the robustness of the system in challenging environments. It is further demonstrated that the integer quantization of the model results in a significant reduction of computation time, especially for the DS-CNN architecture, with less than 5 ms for inference on an embedded CPU and even less than 1 ms on the Edge TPU, with model sizes as low as 0.57 MByte. This shows that the current models can indeed be used with an embedded system, which is an important stepping stone for the adoption of real-time capable KWS for industrial control.

Background noise from a factory scenario has a significant impact on the accuracy. Consequently, the straight forward adoption of this KWS for safety critical tasks is currently unreasonable with an error rate of about 8.70 % in noisy scenarios and about 3.80 % without noise interference. Still, an adoption of this technology as an auxiliary means of control for less critical assistance tasks is feasible.

The robustness of such a system can be improved if a repeat-to-confirm communication pattern is adopted: The KWS repeats the inferred command and the operator confirms it again. As a side effect, this would identify miss-classified commands and allows for the collection of more training data that is specific to the use case environment. Thus, if such a system is adopted, it should be managed in a way that allows updates to the model itself. It is reasonable to expect future training data that includes valuable training samples that previously lead to miss-classification. Additionally, further updates to the model architecture which improve its robustness can be expected. The application of this KWS using more application specific data as well as the performance on other embedded hardware platforms, including the real time calculation of the input features, will be investigated in the future.

ACKNOWLEDGMENT

This work has been achieved in the European ITEA project "OPTimised Industrial IoT and Distributed Control Platform for Manufacturing and Material Handling (OPTIMUM) and has been funded by the German Federal Ministry of Education and Research (BMBF) under reference number 01IS17027. We want to thank all partners in the OPTIMUM project for the stimulating discussions and their contributions to the project. All project partners can be found on https://itea3.org/project/optimum.html.

REFERENCES

[1] S. Cass, "Taking ai to the edge: Google's tpu now comes in a maker-friendly package," *IEEE Spectrum*, vol. 56, no. 5, pp. 16–17, 2019.
[2] Coral.ai products page. Accessed on: Dec. 03, 2020. [Online]. Available: https://coral.ai/products/
[3] S. Boruah and S. Basishtha, "A study on hmm based speech recognition system," in *2013 IEEE International Conference on Computational Intelligence and Computing Research*. IEEE, 2013, pp. 1–5.
[4] G. Chen, C. Parada, and G. Heigold, "Small-footprint keyword spotting using deep neural networks," in *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2014, pp. 4087–4091.
[5] T. N. Sainath and C. Parada, "Convolutional neural networks for small-footprint keyword spotting," in *Sixteenth Annual Conference of the International Speech Communication Association*, 2015.
[6] S. K. Gouda, S. Kanetkar, D. Harrison, and M. K. Warmuth, "Speech recognition: Keyword spotting through image recognition," *arXiv preprint arXiv:1803.03759*, 2018.
[7] M. Ravanelli and Y. Bengio, "Speech and speaker recognition from raw waveform with sincnet," *arXiv preprint arXiv:1812.05920*, 2018.
[8] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 1251–1258.
[9] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
[10] X. Zhang, X. Zhou, M. Lin, and J. Sun, "Shufflenet: An extremely efficient convolutional neural network for mobile devices," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 6848–6856.
[11] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello edge: Keyword spotting on microcontrollers," *arXiv preprint arXiv:1711.07128*, 2017.
[12] S. Choi, S. Seo, B. Shin, H. Byun, M. Kersner, B. Kim, D. Kim, and S. Ha, "Temporal convolution for real-time keyword spotting on mobile devices," *arXiv preprint arXiv:1904.03814*, 2019.
[13] A. Saade, A. Coucke, A. Caulier, J. Dureau, A. Ball, T. Bluche, D. Leroy, C. Doumouro, T. Gisselbrecht, F. Caltagirone *et al.*, "Spoken language understanding on the edge," *arXiv preprint arXiv:1810.12735*, 2018.
[14] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *arXiv preprint arXiv:1804.03209*, 2018.
[15] B. Zhen, X. Wu, Z. Liu, and H. Chi, "On the importance of components of the mfcc in speech and speaker recognition," in *Sixth International Conference on Spoken Language Processing*, 2000.
[16] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2017.
[17] D. Masters and C. Luschi, "Revisiting small batch training for deep neural networks," *arXiv preprint arXiv:1804.07612*, 2018.
[18] ARM Compiler armcc User Guide: The NEON unit. Accessed on: Dec. 03, 2020. [Online]. Available: https://developer.arm.com/documentation/dui0472/m/using-the-neon-vectorizing-compiler/the-neon-unit